Linux OS and Scripting

Name: Ananya H C

Register Number: 261100690015

1. Hard Link and Soft Link

1.1 Objective

To understand and demonstrate the creation, identification, behavior, and differences between hard links and symbolic links in Linux.

The practical also demonstrates inode numbers, link counts, symbolic link targets, file deletion, directory linking, and cross-filesystem link behavior.

1.2 Question 1: Create a Working Directory

Question:

Create a directory named `link_lab` in the home directory and use it as the working directory.

Command:

```
cd ~
mkdir link_lab
cd link_lab
pwd
```

Explanation:

The `cd` command moves to the user's home directory. The `mkdir` command creates the directory `link_lab`. The final `cd` command enters the newly created directory. The `pwd` command displays the current working directory.

Observation:

The working directory is successfully created and entered.

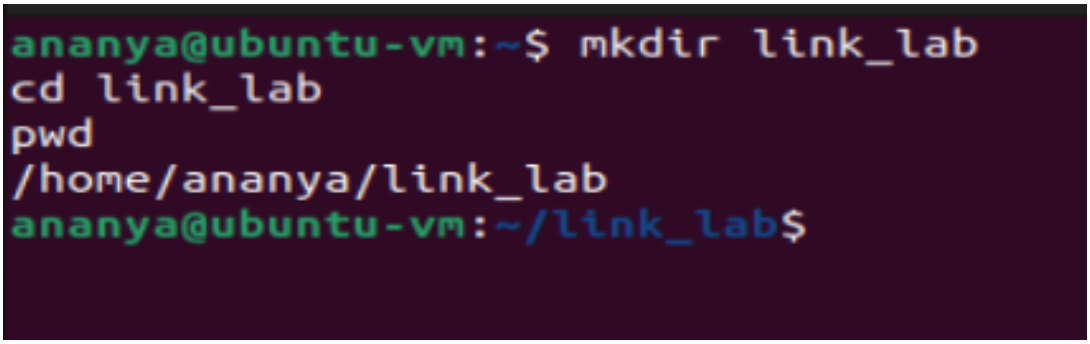


Figure 1: Creation of the link_lab working directory.

1.3 Question 2: Create the Original File

Question:

Create a file named `security.txt` containing security-related information.

Command:

```
cat > security.txt << 'EOF'
Security alert: Unauthorized login attempt detected.
System monitoring is active.
EOF

cat security.txt
```

Explanation:

The `cat` command with a here-document is used to create `security.txt`. The text entered between `EOF` markers is stored in the file.

The second `cat` command displays the contents of the file.

Observation:

The original security file is successfully created.

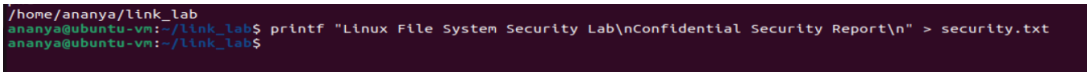


Figure 2: Creation and display of security.txt.

1.4 Question 3: Create Hard and Symbolic Links

Question:

Create one hard link and one symbolic link for `security.txt`.

Command:

```
ln security.txt security_hard.txt
ln -s security.txt security_soft.txt
```

```
ls -li
```

Explanation:

The `ln` command creates a hard link named `security_hard.txt`. A hard link refers directly to the same inode as the original file.

The `ln -s` command creates a symbolic link named `security_soft.txt`. A symbolic link stores a path referring to another file.

Observation:

The hard link and symbolic link are successfully created.

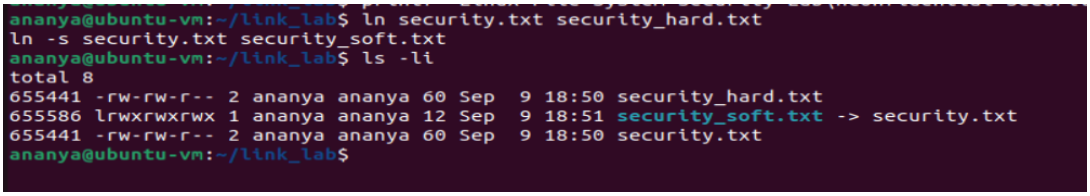


Figure 3: Creation of the hard link and symbolic link.

1.5 Question 4: Examine Link Metadata

Question:

Use `ls -l` to examine the metadata of the original file, hard link, and symbolic link.

Command:

```
ls -l security.txt security_hard.txt security_soft.txt
```

Explanation:

The `ls -l` command displays detailed file information. A symbolic link is identified by the letter `l` at the beginning of its permissions and normally displays an arrow pointing to its target.

Observation:

The symbolic link is displayed as a link to `security.txt`, while the hard link appears as a normal file.

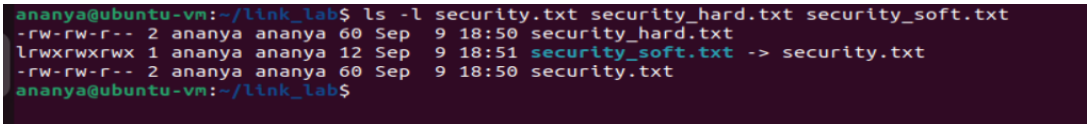


Figure 4: Detailed metadata of the original file, hard link, and symbolic link.

1.6 Question 5: Compare Inodes and Link Counts

Question:

Use `stat` to compare the inode number, file size, and link count of the original file, hard link, and symbolic link.

Command:

```
stat security.txt
stat security_hard.txt
stat security_soft.txt
```

Explanation:

The `stat` command displays detailed information about a file.

The original file and its hard link refer to the same inode. Therefore, they have the same inode number and share the same link count.

The symbolic link has its own inode because it is a separate filesystem object containing a reference to the target.

Observation:

The original file and hard link share the same inode, whereas the symbolic link has a different inode.



Figure 5: Comparison of file metadata using stat.

1.7 Question 6: Compare readlink Behavior

Question:

Use `readlink` to determine which object is a symbolic link.

Command:

```
readlink security_soft.txt
readlink security_hard.txt
```

Explanation:

The `readlink` command displays the target of a symbolic link. It does not produce output for an ordinary file or hard link because a hard link is not a path-based link object.

Observation:

The symbolic link displays `security.txt`, while the hard link does not produce a target path.

```
ananya@ubuntu-vm:~/link_lab$ readlink security_soft.txt
readlink security_hard.txt
security.txt
ananya@ubuntu-vm:~/link_lab$
```

Figure 6: Comparison of readlink output for symbolic and hard links.

1.8 Question 7: Find the Full Target Path

Question:

Find the full path to the target of the symbolic link.

Command:

```
readlink -f security_soft.txt
```

Explanation:

The `-f` option resolves the symbolic link completely and displays the absolute path of the final target.

Observation:

The command displays the absolute path of `security.txt`.

```
ananya@ubuntu-vm:~/link_lab$ readlink -f security_soft.txt
/home/ananya/link_lab/security.txt
ananya@ubuntu-vm:~/link_lab$
```

Figure 7: Full path of the symbolic link target.

1.9 Question 8: Access Content Through All Links

Question:

Display the contents of the original file, hard link, and symbolic link.

Command:

```
cat security.txt
cat security_hard.txt
cat security_soft.txt
```

Explanation:

The original file and hard link refer to the same inode. The symbolic link refers to the original file by path. Therefore, all three commands display the same file contents while the target exists.

Observation:

The same security information is accessible through all three names.

```
ananya@ubuntu-vm:~/link_lab$ echo "Unauthorized login detected" >> security.txt
ananya@ubuntu-vm:~/link_lab$ echo "----- security.txt -----"
cat security.txt

echo "----- security_hard.txt -----"
cat security_hard.txt

echo "----- security_soft.txt -----"
cat security_soft.txt
----- security.txt -----
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
----- security_hard.txt -----
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
----- security_soft.txt -----
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
ananya@ubuntu-vm:~/link_lab$
```

Figure 8: File content accessed through the original file, hard link, and symbolic link.

1.10 Question 9: Modify the File Through the Hard Link

Question:

Modify the file through the hard link and verify the change through the other links.

Command:

```
echo "Unauthorized login detected" >> security.txt

cat security_hard.txt
cat security_soft.txt

echo "Modified through hard link" >> security_hard.txt

cat security.txt
cat security_soft.txt
```

Explanation:

Because the hard link and original file refer to the same inode, modifying the file through either name modifies the same underlying file.

The symbolic link also reflects the modification because it points to the same target file.

Observation:

Changes made through the hard link are visible through both the original filename and the symbolic link.



Figure 9: Modification of file contents through the hard link.

1.11 Question 10: Rename the Original File

Question:

Rename `security.txt` to `incident.txt` and observe the behavior of the hard and symbolic links.

Command:

```
mv security.txt incident.txt

cat security_hard.txt
cat security_soft.txt
```

Explanation:

Renaming the original filename does not affect the inode or the hard link. Therefore, `security_hard.txt` continues to access the file.

The symbolic link still contains the old path `security.txt`. Since that path no longer exists, the symbolic link becomes broken.

Observation:

The hard link continues to work after the original filename is renamed, while the symbolic link becomes invalid.

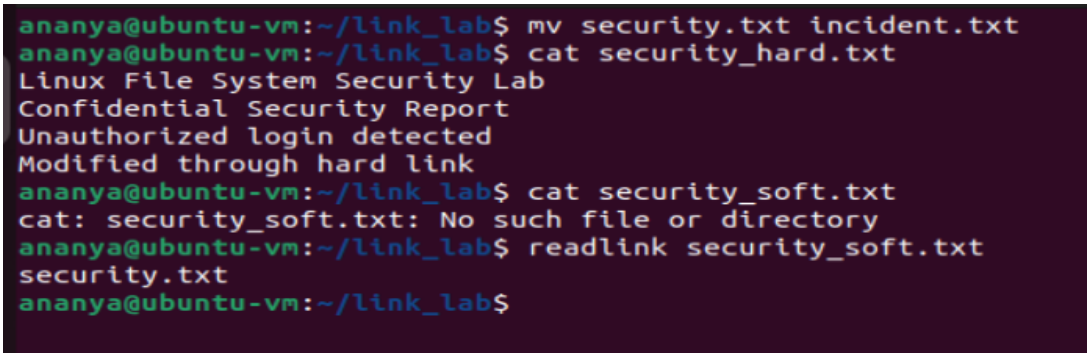


Figure 10: Behavior of hard and symbolic links after renaming the original file.

1.12 Question 11: Recreate the Symbolic Link

Question:

Create a new symbolic link pointing to `incident.txt`.

Command:

```
rm security_soft.txt
ln -s incident.txt security_soft.txt

readlink security_soft.txt
cat security_soft.txt
```

Explanation:

The old broken symbolic link is removed first. A new symbolic link is then created with `incident.txt` as its target.

`readlink` confirms the new target and `cat` verifies that the target contents can be accessed.

Observation:

The symbolic link is recreated successfully and points to `incident.txt`.

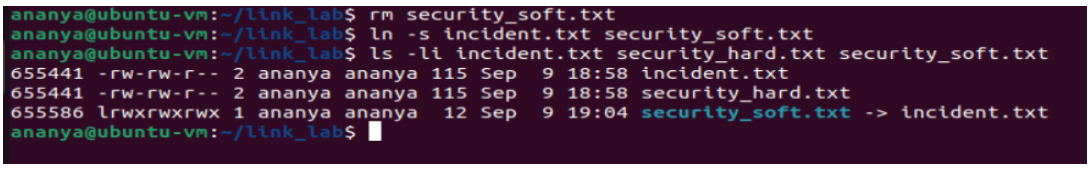


Figure 11: Recreation of the symbolic link pointing to incident.txt.

1.13 Question 12: Delete the Original File and Check the Hard Link

Question:

Delete `incident.txt` and check whether the hard link still provides access to the file.

Command:

```
rm incident.txt

cat security_hard.txt
```

Explanation:

Deleting a filename does not necessarily delete the underlying data. The data remains accessible as long as another hard link refers to the inode.

Since `security_hard.txt` is another hard link, the file contents remain accessible through it.

Observation:

The hard link continues to work even after the original filename has been deleted.

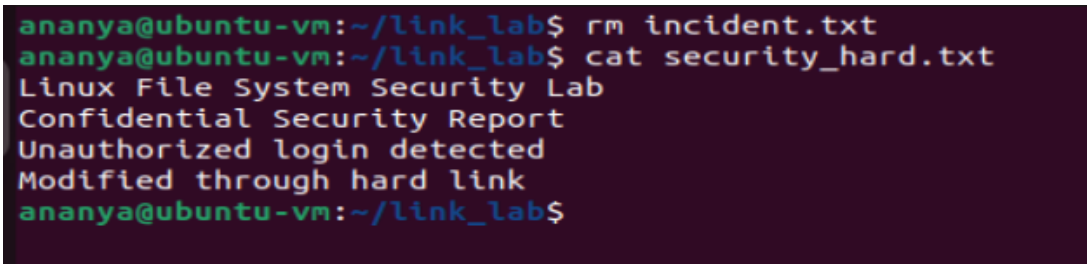


Figure 12: Hard link access after deletion of the original filename.

1.14 Question 13: Observe the Broken Symbolic Link

Question:

Check the symbolic link after its target file has been deleted.

Command:

```
readlink security_soft.txt
cat security_soft.txt
```

Explanation:

The symbolic link contains a path to its target. Deleting the target does not automatically delete the symbolic link. Instead, the link remains but points to a nonexistent path.

Observation:

The symbolic link is broken because `incident.txt` no longer exists.

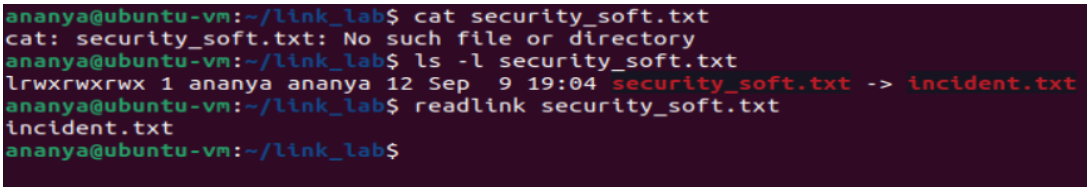


Figure 13: Broken symbolic link after deletion of its target.

1.15 Question 14: Attempt to Create a Hard Link to a Directory

Question:

Attempt to create a hard link to a directory.

Command:

```
mkdir evidence
ln evidence evidence_hard
```

Explanation:

Linux normally does not allow ordinary users to create hard links to directories. This restriction helps prevent filesystem structures such as directory cycles.

Therefore, the `ln` command is expected to fail.

Observation:

The attempt to create a hard link to the directory fails as expected.

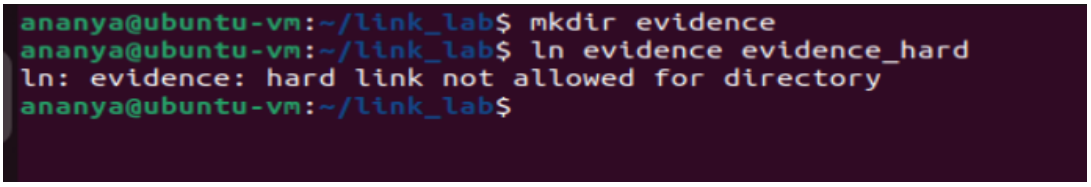


Figure 14: Attempt to create a hard link to a directory.

1.16 Question 15: Create a Symbolic Link to a Directory

Question:

Create a symbolic link to the `evidence` directory.

Command:

```
ln -s evidence evidence_link
ls -l evidence_link
```

Explanation:

Unlike hard links to directories, symbolic links can point to directories. The symbolic link stores the path of the directory as its target.

Observation:

The symbolic link to the directory is created successfully.

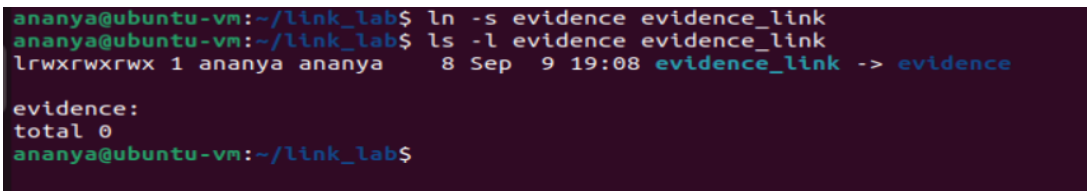


Figure 15: Creation of a symbolic link to a directory.

1.17 Question 16: Check Available Filesystems

Question:

Display the mounted filesystems and identify the filesystems available for testing cross-filesystem links.

Command:

```
df -T
```

Explanation:

The `df -T` command displays filesystem usage together with filesystem type information.

The output can be used to identify different mounted filesystems, such as the Linux ext4 filesystem and other mounted filesystems.

Observation:

The mounted filesystem information is displayed, including the filesystem types available on the system.

```
ananya@ubuntu-vm:~/link_lab$ df -T
Filesystem     Type     1K-blocks     Used Available Use% Mounted on
tmpfs          tmpfs       396124     2436    393688   1% /run
/dev/sda3      ext4      40453376 19069440  19296820  50% /
tmpfs          tmpfs      1980616        0   1980616   0% /dev/shm
tmpfs          tmpfs         5120        4      5116   1% /run/lock
/dev/sda2      vfat        524252     6232    518020   2% /boot/efi
tmpfs          tmpfs       396120      108    396012   1% /run/user/1001
/dev/sr1       iso9660    4919592  4919592         0 100% /media/ananya/Ubuntu 22.04.3 LTS amd64
/dev/fd0       vfat          1424        9      1415   1% /media/floppy0
ananya@ubuntu-vm:~/link_lab$
```

Figure 16: Mounted filesystems and their filesystem types.

1.18 Question 17: Attempt a Hard Link Across Filesystems

Question:

Attempt to create a hard link from a file on the home filesystem to the mounted floppy filesystem.

Command:

```
echo "Cross filesystem test" > cross_test.txt

ln cross_test.txt /media/floppy0/cross_hard.txt
```

Explanation:

Hard links require the source and destination to be on the same filesystem because the hard link refers directly to an inode.

When the destination is located on a different filesystem, Linux rejects the operation.

Observation:

The hard-link operation fails with an `Invalid cross-device link` error.

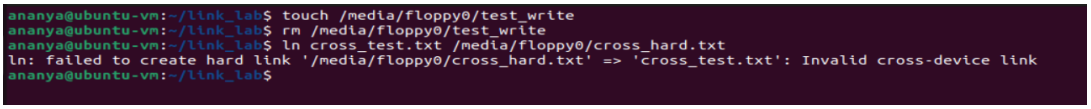


Figure 17: Failed attempt to create a hard link across filesystems.

1.19 Question 18: Create a Symbolic Link Across Filesystems

Question:

Attempt to create a symbolic link to the file on the mounted floppy filesystem.

Command:

```
ln -s /home/ananya/link_lab/cross_test.txt /media/floppy0/cross_soft.txt
```

Explanation:

Symbolic links store a pathname and are not inherently restricted to the same filesystem as their target.

However, the destination filesystem used in this experiment is a VFAT filesystem. VFAT does not support creation of Unix symbolic link objects in the usual way. Therefore, the operation is rejected by the filesystem.

Observation:

The symbolic-link creation attempt fails with an `Operation not permitted` error because the destination filesystem does not support creating the required symbolic link.

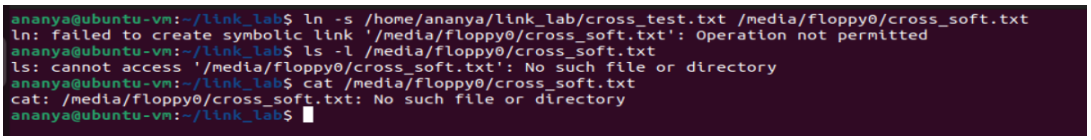


Figure 18: Attempt to create a symbolic link on the mounted VFAT filesystem.

2. Comparison of Hard Links and Symbolic Links

Feature	Hard Link	Symbolic Link
Inode	Shares the same inode as the original file	Has a separate inode
Reference	Direct reference to filesystem inode	Stores a pathname to the target
Cross-filesystem	Cannot normally cross filesystems	Can refer to a target on another filesystem, subject to destination filesystem support
Directory linking	Normally not allowed for ordinary users	Can point to directories
Original filename deleted	Hard link continues to work	Link becomes broken if the target pathname no longer exists
File size	Represents the actual file data	Represents the link object and stored path
Identification	Appears as a normal file	Identified as a symbolic link by `ls -l`

3. Sort Command

3.1 Objective

To understand and demonstrate the Linux `sort` command and its use with related commands such as `cut`, `uniq`, `awk`, `grep`, and pipelines for processing security-related data.

3.2 Question 19: Create alerts.txt

Question:

Suppose `alerts.txt` contains:

```
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
```

The format is:

```
IP-address username failed-attempts
```

Create the file.

Command:

```
cd ~
mkdir sort_lab
cd sort_lab

cat > alerts.txt << 'EOF'
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
EOF

cat alerts.txt
```

Explanation:

The commands create a separate working directory named `sort_lab` and create the `alerts.txt` file containing four alert records.

Observation:

The alert records are successfully created and displayed.

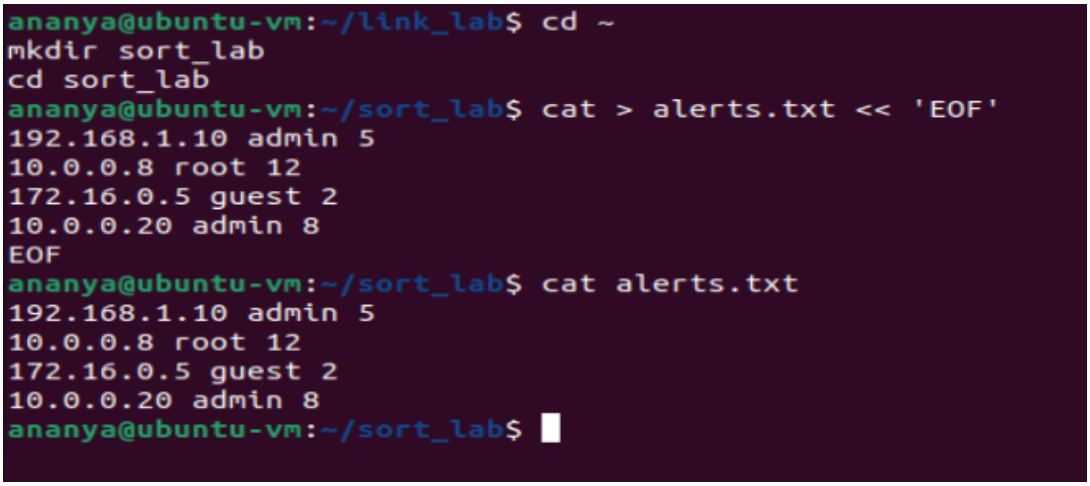


Figure 19: Creation and display of alerts.txt.

3.3 Question 20: Sort According to Failed Attempts

Question:

Sort the records according to the number of failed attempts.

Command:

```
sort -k3,3n alerts.txt
```

Explanation:

The third field contains the number of failed attempts. The `-k3,3` option selects this field and the `n` option performs numerical sorting.

Expected order:

```
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

Observation:

The records are arranged from the lowest to the highest number of failed attempts.

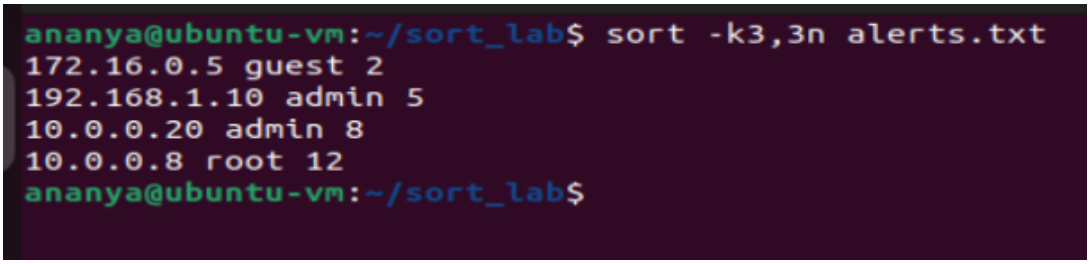


Figure 20: Alert records sorted according to failed attempts.

3.4 Question 21: Sort Suspicious IP Addresses Alphabetically

Question:

Sort a list of suspicious IP addresses alphabetically.

Command:

```
cut -d ' ' -f1 alerts.txt | sort
```

Explanation:

The `cut` command extracts the first field of each record. The output is then passed to `sort` using a pipe.

Expected output:

```
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Observation:

The IP addresses are displayed in sorted order.

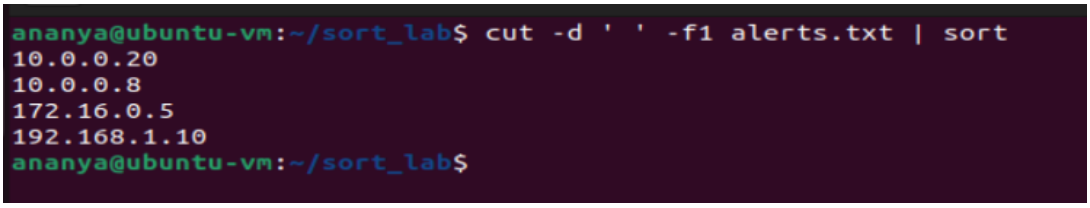


Figure 21: Suspicious IP addresses sorted alphabetically.

3.5 Question 22: Display Only Unique IP Addresses

Question:

Display only unique suspicious IP addresses using `sort`.

Command:

```
printf "192.168.1.10\n10.0.0.8\n192.168.1.10\n172.16.0.5\n10.0.0.8\n" >
    suspicious_ips.txt

sort suspicious_ips.txt | uniq
```

Explanation:

The file contains duplicate IP addresses. The `sort` command places equal addresses together, allowing `uniq` to remove duplicate consecutive entries.

Expected output:

```
10.0.0.8
172.16.0.5
192.168.1.10
```

Observation:

Only unique IP addresses are displayed.

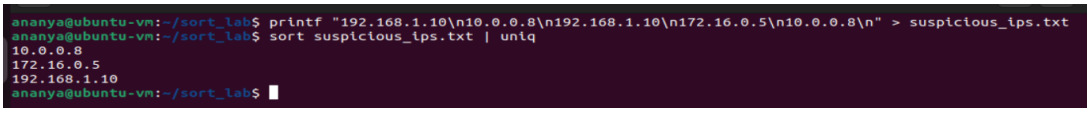


Figure 22: Unique suspicious IP addresses after sorting and duplicate removal.

3.6 Question 23: Sort Port Numbers Numerically

Question:

Sort a list of detected port numbers numerically.

Command:

```
printf "22\n443\n80\n8080\n21\n" > ports.txt

sort -n ports.txt
```

Explanation:

The `-n` option causes the values to be compared numerically rather than alphabetically.

Expected output:

```
21
22
80
443
8080
```

Observation:

The port numbers are arranged in ascending numerical order.



Figure 23: Port numbers sorted numerically in ascending order.

3.7 Question 24: Display Port Numbers in Descending Order

Question:

Display port numbers in descending numerical order.

Command:

```
sort -nr ports.txt
```

Explanation:

The `-n` option performs numerical sorting, while `-r` reverses the sorting order.

Expected output:

```
8080
443
80
22
21
```

Observation:

The largest port number appears first and the smallest appears last.

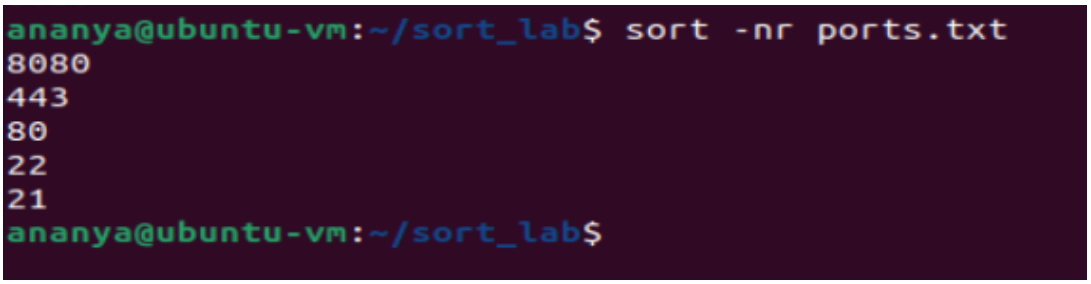


Figure 24: Port numbers sorted in descending numerical order.

3.8 Question 25: Sort Usernames Without Considering Case

Question:

Sort usernames without considering uppercase and lowercase differences.

Command:

```
printf "Admin\nroot\nGUEST\nadmin\nUser\n" > usernames.txt

sort -f usernames.txt
```

Explanation:

The `-f` option tells `sort` to fold lowercase and uppercase characters together when comparing values.

Expected output:

```
Admin
admin
GUEST
root
User
```

Observation:

The usernames are sorted without treating uppercase and lowercase characters as different for comparison.

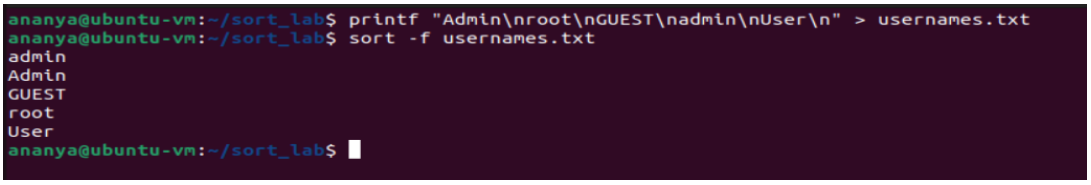


Figure 25: Usernames sorted without considering case differences.

3.9 Question 26: Count IP Address Frequency

Question:

From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

Command:

```
printf "192.168.1.10 failed\n10.0.0.8 failed\n192.168.1.10 failed\n172.16.0.5
   failed\n10.0.0.8 failed\n192.168.1.10 failed\n10.0.0.8 failed\n" >
   failed_login.log

awk '{print $1}' failed_login.log | sort | uniq -c | sort -nr
```

Explanation:

The pipeline performs the following operations:

1. `awk` extracts the first field containing the IP address.

2. `sort` groups identical IP addresses.

3. `uniq -c` counts the occurrences.

4. `sort -nr` sorts the counts numerically in descending order.

Expected output:

```
3 192.168.1.10
3 10.0.0.8
1 172.16.0.5
```

Observation:

The IP addresses are displayed together with their occurrence counts, with the highest frequency appearing first.

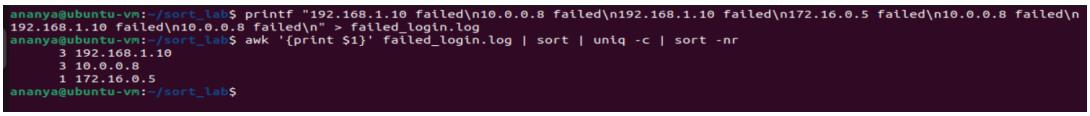


Figure 26: IP address frequency calculated from the failed-login log.

3.10 Question 27: Sort CSV Records by Failed Attempts

Question:

Given records in the form `IP,username,failed_attempts`, sort the records based on the number of failed attempts.

Command:

```
printf "IP,username,failed_attempts\n192.168.1.10,admin,5\n10.0.0.8,root,12\
    n172.16.0.5,guest,2\n10.0.0.20,admin,8\n" > login_records.csv

{ head -n 1 login_records.csv; tail -n +2 login_records.csv | sort -t',' -k3,3
    n; }
```

Explanation:

The `-t','` option specifies that the comma is the field separator. The third field contains the failed-attempt count.

The header is kept at the top using `head`, while the remaining records are sorted numerically using `sort`.

Expected output:

```
IP,username,failed_attempts
172.16.0.5,guest,2
192.168.1.10,admin,5
10.0.0.20,admin,8
10.0.0.8,root,12
```

Observation:

The CSV records are sorted according to the number of failed attempts.



Figure 27: CSV records sorted according to failed attempts.

3.11 Question 28: Sort a Vulnerability Report by Severity

Question:

Sort a CSV vulnerability report based on its severity score.

Command:

```
printf "CVE,Description,SeverityScore\nCVE-2024-1001,SQL Injection,9.8\nCVE
    -2024-1002,Cross Site Scripting,6.5\nCVE-2024-1003,Weak Password,4.3\nCVE
    -2024-1004,Remote Code Execution,10.0\n" > vulnerabilities.csv

{ head -n 1 vulnerabilities.csv; tail -n +2 vulnerabilities.csv | sort -t',' -
    k3,3n; }
```

Explanation:

The severity score is stored in the third field of the CSV file. The `-t','` option specifies the comma delimiter and `-k3,3n` performs numerical sorting based on the third field.

Expected output:

```
CVE,Description,SeverityScore
CVE-2024-1003,Weak Password,4.3
CVE-2024-1002,Cross Site Scripting,6.5
CVE-2024-1001,SQL Injection,9.8
CVE-2024-1004,Remote Code Execution,10.0
```

Observation:

The vulnerability records are arranged from the lowest to the highest severity score.

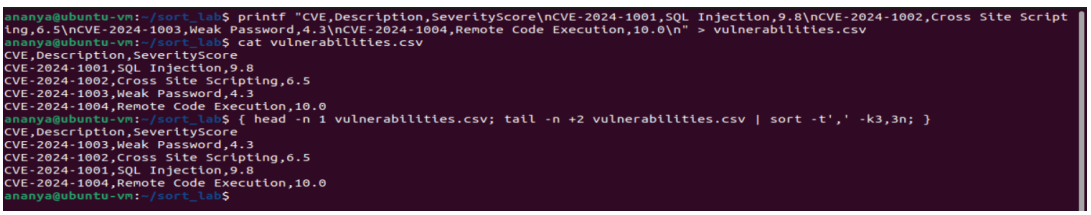


Figure 28: Vulnerability report sorted according to severity score.

3.12 Question 29: Extract Unique IP Addresses Using grep

Question:

Extract IP addresses using `grep`, then use `sort` and `uniq` to identify unique IP addresses.

Command:

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' failed_login.log | sort | uniq
```

Explanation:

The `grep -Eo` command uses an extended regular expression to extract IPv4 addresses from the log.

The extracted IP addresses are passed through `sort`, and `uniq` removes duplicate addresses.

Expected output:

```
10.0.0.8
172.16.0.5
192.168.1.10
```

Observation:

The unique IP addresses are extracted from the failed-login log and displayed in sorted order.

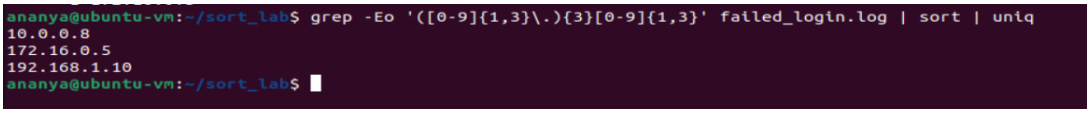


Figure 29: Unique IP addresses extracted using grep, sort, and uniq.

3.13 Question 30: Display the Top 5 IP Addresses

Question:

Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

Command:

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' failed_login.log | sort | uniq -c |
    sort -nr | head -n 5
```

Explanation:

The pipeline performs the following operations:

1. `grep -Eo` extracts IPv4 addresses from the log.
2. `sort` groups identical IP addresses.
3. `uniq -c` counts the occurrences of each IP address.

4. `sort -nr` sorts the counts from highest to lowest.

5. `head -n 5` displays the first five results.

The demonstration log contains only three unique IP addresses. Therefore, only three entries are displayed even though the command requests the top five.

Expected output:

```
3 192.168.1.10
3 10.0.0.8
1 172.16.0.5
```

Observation:

The pipeline successfully identifies the IP addresses associated with the highest number of failed login attempts.

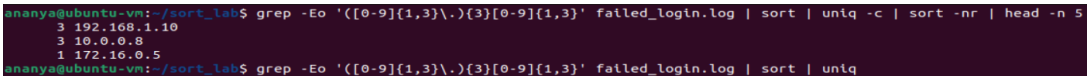


Figure 30: Top IP addresses responsible for failed login attempts.